

PYTHON FOUNDATION CHALLENGE

Revision + Progression (week 1 and 2 + week 3)

Objective: Strengthen foundations, expose gaps, and develop problem-solving ability before OOP by revisiting last week's exercises with advanced constraints, and tackling new algorithmic challenges.

Instructions

- Solve the problems independently. Do not look at previous solutions.
- Focus on producing a working solution first; clean up the code afterward if time allows.
- No external libraries unless a question explicitly permits them.
- If you get stuck, identify the Python concept causing the problem rather than immediately searching for a solution.

Part A — Previous Questions: Revision

Redo these from memory. The purpose is to check retention before moving into the new challenges.

1. Simple Calculator

Suggested time: 20–25 minutes

Problem

Build a continuous calculator that maintains a running total. The user inputs an operator and a number repeatedly (e.g., + 5, then * 2).

Examples

Input: + 5 → Result: 5

Input: * 2 → Result: 10

Input: undo → Result: 5

Constraints

- Must handle floating-point numbers.
- Implement an undo feature that reverts the calculation to the previous state.
- Strictly forbid the use of `eval()`.
- Must handle division by zero and invalid operators without crashing the program.
- Do not import libraries.

2. Palindrome

Suggested time: 20–25 minutes

Problem

Write a program that determines whether a full sentence reads the same forwards and backwards, ignoring mixed casing, spaces, and punctuation (e.g., *"Was it a car or a cat I saw?"* is a palindrome).

Examples

Input: Was it a car or a cat I saw? → Output: Palindrome

Input: Hello, World! → Output: Not a palindrome

Constraints

- Must be implemented using a strict two-pointer approach (left and right indices moving inward).
- 7. You are not allowed to create any new reversed strings, substrings, or use built-in string reversal methods (like `[::-1]` or `.reverse()`).
- You must manually skip spaces and punctuation during the pointer comparison.
- Do not import libraries.

3. Student Grades

Suggested time: 20–25 minutes

Problem

Given a list of dictionaries representing students and their scores, calculate each student's average and assign a letter grade.

Examples

```
students = [{"name": "Sam", "scores": [80, 90]}, {"name": "David", "scores": [55, 60]}
```

Sam → Average: 85.00 → Grade: A

David → Average: 57.50 → Grade: C

Grading System

70–100 → A
60–69 → B
50–59 → C
45–49 → D
40–44 → E

Constraints

- Calculate the average manually (do not use `sum()` or `statistics.mean()`).
- As you loop through the students, manually track and print the highest and lowest performing students.
- You are forbidden from using `max()` or `min()` on the averages; you must track the running highest/lowest using standard variables and `if` statements.
- Use the grading system from Week 1 (70-100: A, 60-69: B, etc.).

4. Second Largest

Suggested time: 15–20 minutes

Problem

Given a list of numbers, find the second-largest unique number.

Examples

Input: [10, 5, 8, 20, 15] → Output: 15

Input: [4, 9, 2, 9, 7] → Output: 7

Constraints

- You are strictly forbidden from using `sort()`, `sorted()`, `max()`, or converting the list to a `set()`.
- You must track the `largest` and `second_largest` variables manually and update them in a single pass (one loop) through the list.
- Duplicate values must not affect the result.

5. Word Frequency

Suggested time: 15–20 minutes

Problem

Given a paragraph of text, count how many times each word appears and store it in a dictionary. Then, find and print the single most frequent word.

Examples

Input: "Python is fun, and Python is powerful!"

Output: Most frequent word: python (2)

Constraints

- Treat uppercase and lowercase as the same.
- You must strip basic punctuation (like commas and periods) manually without using the `re` module or `string.punctuation`.
- After counting, you must find the most frequent word without using `max()` or sorting the dictionary.
- Do not use `collections.Counter`.

6. Two Sum

Suggested time: 20–25 minutes

Problem

Given a list of numbers and a target number, find the first valid pair of numbers whose sum equals the target. Return their indices (positions).

Examples

Numbers: [2, 7, 11, 15] | Target: 9 → Output: [0, 1]

Constraints

- You must return the indices of the two numbers, not the numbers themselves.
- You must achieve $O(n)$ time complexity by using a dictionary to track the numbers you've seen so far.
- Nested loops ($O(n^2)$) are strictly forbidden.
- The two numbers must come from different positions in the list.

Part B — New Questions: Progression

These are new problems designed to increase the difficulty and reveal gaps in core Python problem-solving skills.

7. Anagram Finder

Suggested time: 15–20 minutes

Problem

Check if two words are anagrams of each other (meaning they contain the exact same characters in the exact same frequencies, just in a different order).

Examples

Input: Word 1: listen | Word 2: silent → Output: Anagram

Input: Word 1: triangle | Word 2: integral → Output: Anagram

Input: Word 1: apple | Word 2: pale → Output: Not an anagram

Input: Word 1: hello | Word 2: billion → Output: Not an anagram

Constraints

- Do not use `sorted()` or `collections.Counter`.
- You must build a character frequency map manually (using a dictionary) to compare the letters.
- Treat uppercase and lowercase as the same (e.g., `Triangle` and `integral` should match).
- Do not import external libraries.

7. Caesar Cipher

Suggested time: 20–25 minutes

Problem

Build a program that can encrypt and decrypt a message by shifting letters by a given number. For example, a shift of 1 makes 'a' become 'b', and 'z' becomes 'a'.

Examples

Input: Text: hello | Shift: 1 | Mode: encrypt → Output: ifmmp

Input: Text: xyz | Shift: 2 | Mode: encrypt → Output: zab (*Notice the wrap-around*)

Input: Text: hello, world! | Shift: 3 | Mode: encrypt → Output: khoor, zruog! (*Notice punctuation and spaces are preserved*)

Input: Text: ifmmp | Shift: 1 | Mode: decrypt → Output: hello

Constraints

- Must handle alphabetical wrap-around correctly (e.g., 'z' shifted by 1 becomes 'a', 'a' shifted backward by 1 becomes 'z').
- Must preserve spaces, numbers, and punctuation without shifting or altering them.
- The program must ask the user if they want to encrypt or decrypt.

- Do not use external libraries (like `cryptography`).

7. Number Reverser

Suggested time: 15–20 minutes

Problem

Reverse a given integer. For example, 12345 becomes 54321.

Examples

Input: 12345 → Output: 54321

Input: -987 → Output: -789

Input: 1200 → Output: 21 (*Mathematically, leading zeros are dropped*)

Input: 0 → Output: 0

Constraints

- You are NOT allowed to convert the integer to a string (no `str()`, no string slicing like `[::-1]`).
- You must use pure mathematical operations (specifically modulo `%` to get the last digit, and integer division `//` to remove the last digit).
- Must handle negative numbers correctly (the negative sign should remain at the front).
- Do not import libraries.

Progression

The challenge moves from advanced state management and two-pointer logic → manual algorithmic tracking and nested data processing → hash map optimization and character frequency mapping → pure mathematical manipulation.

Time Guide

Section	Questions	Suggested Time
Revision Advanced	1–2	40–50 min total
Revision (inter-Adv)	3–6	60–70 min total
New progression	7–9	45–60 min total
Overall	1–6	About 2 hrs 30 mins

